

Verificación integral de la solución

Resultados comprobados del sistema de gestión de rutas logísticas.

5.000 filas

658 válidas

4.342 rechazadas

Andrés Felipe García · Evidencias capturadas el 14/09/2026

The screenshot shows a web application for managing routes. The interface includes a header with navigation links, a main content area with a 'Rutas' section, and a table of route data. The table has columns for 'TRAYECTO', 'OFICINA', 'PRIORIDAD', 'DISTANCIA', 'VENTANA', and 'ESTADO'. The data rows show various routes with their respective details and status (Fallida, Ejecutada).

TRAYECTO	OFICINA	PRIORIDAD	DISTANCIA	VENTANA	ESTADO
CII 21 #77-30, Medellín → CII 11 #28-2, Bogotá	CTG-ACDYL ID 722	P0796 ID 796	6.69 km	11/02/2026 01:15 hasta 02:15	Fallida
CII 59 #55-20, Bogotá → CII 26 #23-10, Medellín	CTG-MEDSY ID 255	P0488 ID 488	11.83 km	10/02/2026 23:15 hasta 00:15	Ejecutada
CII 45 #15-12, Medellín → CII 58 #72-1, Cali	BGA-FBQFX ID 29	P0837 ID 837	14.5 km	11/02/2026 00:10 hasta 03:10	Fallida
CII 10 #62-9, Medellín					

Integración frontend, proxy y API verificada

Frontend → proxy → API

The screenshot shows a VS Code editor with a Docker Compose file named `compose.yml` in the `prueba-tecnica-rutas` directory. The file defines a service named `frontend` with the following configuration:

```
services:
  frontend:
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
```

The terminal output shows the execution of the `curl` command from the `frontend` service:

```
(.venv) @andrestatagarcia-droid →/workspaces/prueba-tecnica-rutas/frontend (main) $ curl -i --max-time 15 http://localhost:4200/api/v1/health/
HTTP/1.1 200 OK
vary: origin
date: Mon, 14 Sep 2026 10:43:55 GMT
server: WSGIServer/0.2 CPython/3.14.2
content-type: application/json
x-frame-options: DENY
content-length: 41
x-content-type-options: nosniff
referrer-policy: same-origin
cross-origin-opener-policy: same-origin
connection: keep-alive

{"status": "ok", "service": "routes-api"}(.venv) @andrestatagarcia-droid →/workspaces/prueba-tecnica-rutas/frontend (main) $
```

RESULTADO

La solicitud **GET /api/v1/health/** se realizó desde el puerto 4200 y atravesó correctamente el proxy del frontend.

HALLAZGO

La URL relativa **/api/v1** funcionó con el destino local y quedó parametrizada para usar **backend:8000** dentro de Docker.

EVIDENCIA

Respuesta obtenida: **HTTP 200 OK, status: ok** y **service: routes-api**.

Carga y calidad de datos verificadas

Dataset completo procesado

5.000

filas evaluadas

658

rutas importadas

4.342

rutas rechazadas

5.093

incidencias de regla

RESULTADO

El archivo **data/dataset.xlsx** se procesó completo: 5.000 filas evaluadas, 658 importadas y 4.342 rechazadas.

HALLAZGO

Las seis hojas quedaron conservadas en **staging**. Una fila pudo incumplir varias reglas; por eso se registraron 5.093 incidencias sobre 4.342 rutas rechazadas.

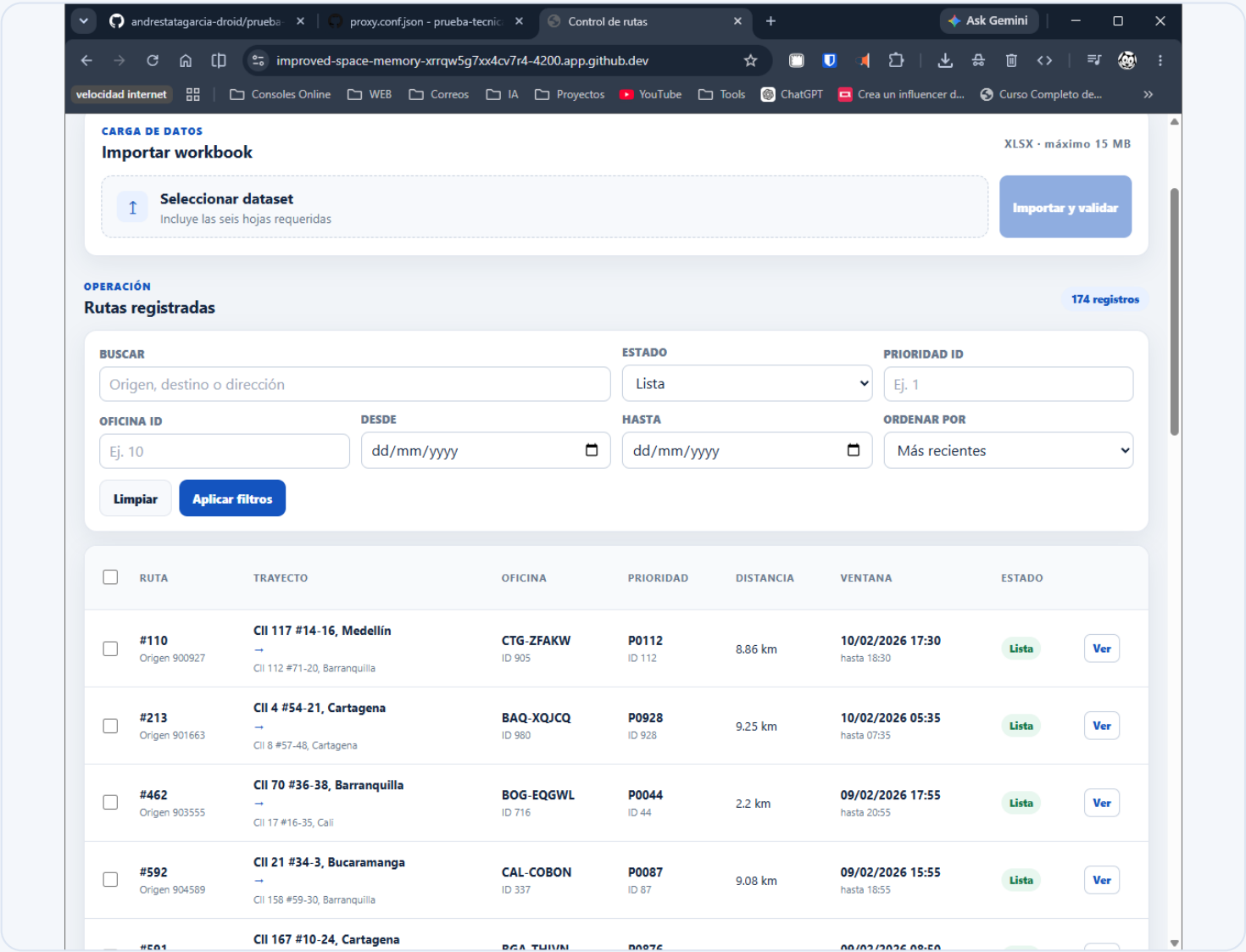
EVIDENCIA

Los controles independientes y las aserciones SQL reprodujeron exactamente los conteos del lote.

```
(.venv) @andrestatagarcia-droid →/workspaces/prueba-tecnica-rutas (main) $ curl -sS -X POST -F "file=@data/dataset.xlsx" http://localhost:8000/api/v1/routes/import/ | python -c "import sys, json; d=json.load(sys.stdin); print(json.dumps({k:d.get(k) for k in ['batch_id', 'status', 'total_rows', 'imported', 'rejected', 'error_count', 'has_more_errors']}), indent=2))"
{
  "batch_id": "493a40e4-c668-477d-8449-d646a487ba18",
  "status": "COMPLETED",
  "total_rows": 5000,
  "imported": 658,
  "rejected": 4342,
  "error_count": 5093,
  "has_more_errors": true
}
(.venv) @andrestatagarcia-droid →/workspaces/prueba-tecnica-rutas (main) $
```

Consulta y filtros operacionales verificados

Estado READY / Lista



RESULTADO

El filtro de estado **Lista** devolvió 174 rutas antes de la ejecución registrada. También quedaron disponibles búsqueda, prioridad, oficina, fechas y ordenamiento.

HALLAZGO

El listado paginado mantuvo el contrato del API y la interfaz restringió la selección a registros **READY**.

EVIDENCIA

El total operacional persistido fue de 658 rutas y el subconjunto **READY** observado fue de 174.

HALLAZGO

El backend confirmó el estado **READY** dentro de una transacción y actualizó la ruta a **EXECUTED**.

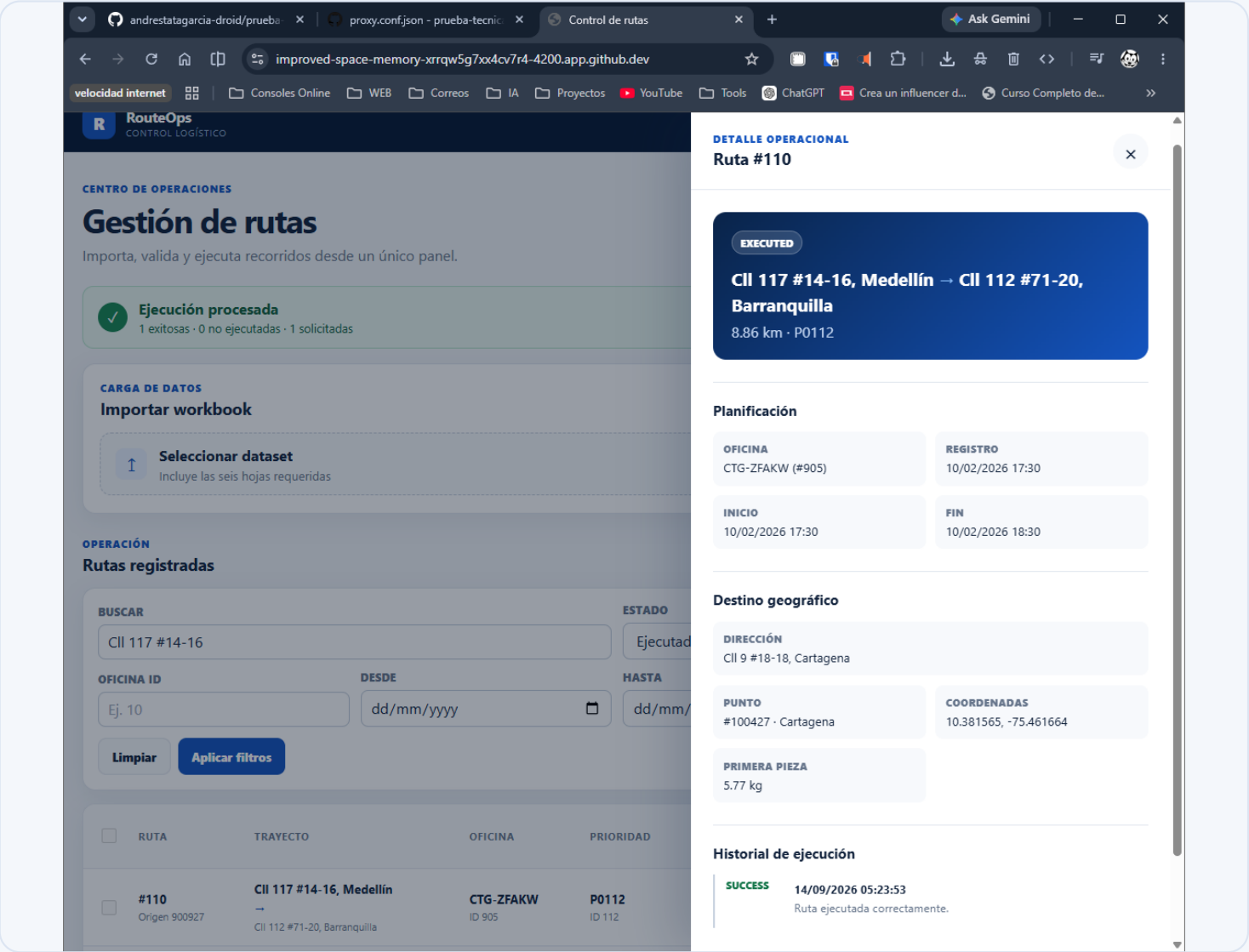
EVIDENCIA

La alerta registró 1 exitosa, 0 no ejecutadas y 1 solicitada. El contador READY bajó de **174 a 173**.

Sistema de gestión de rutas logísticas · Evidencia real de ejecución

Detalle e historial verificados

Trazabilidad por ruta



RESULTADO

El detalle de la ruta **#110** presentó planificación, destino geográfico e historial de ejecución en un único panel.

HALLAZGO

La ruta quedó en estado **EXECUTED** y el evento **SUCCESS** conservó fecha, resultado y mensaje.

EVIDENCIA

La evidencia incluye oficina, ventana, coordenadas, peso de primera pieza y el log asociado.

Resultados múltiples verificados

Respuesta independiente por identificador

Sistema de gestión de rutas logísticas

```

(.venv) @andrestatagarcia-droid → /workspaces/prueba-tecnica-rutas (main) $ curl -sS
-X POST -H "Content-Type: application/json" -d '{"route_id": [616, 192, 999999]}' http
p://localhost:8080/api/v1/routes/execute/ | python -m json.tool
{
  "requested": 3,
  "executed": 1,
  "failed": 2,
  "results": [
    {
      "route_id": 616,
      "result": "SUCCESS",
      "status": "EXECUTED",
      "message": "Ruta ejecutada correctamente."
    },
    {
      "route_id": 192,
      "result": "ERROR",
      "status": "EXECUTED",
      "message": "La ruta no est\u00e1 en estado READY."
    },
    {
      "route_id": 999999,
      "result": "NOT_FOUND",
      "status": null,
      "message": "La ruta no existe."
    }
  ]
}
  
```

Sistema de gestión de rutas logísticas

```

(.venv) @andrestatagarcia-droid → /workspaces/prueba-tecnica-rutas (main) $ curl -sS
-X POST -H "Content-Type: application/json" -d '{"route_id": [616, 192, 999999]}' ht
p://localhost:8080/api/v1/routes/execute/ | python -m json.tool
{
  "requested": 3,
  "executed": 1,
  "failed": 2,
  "results": [
    {
      "route_id": 616,
      "result": "SUCCESS",
      "status": "EXECUTED",
      "message": "Ruta ejecutada correctamente."
    },
    {
      "route_id": 192,
      "result": "ERROR",
      "status": "EXECUTED",
      "message": "La ruta no est\u00e1 en estado READY."
    },
    {
      "route_id": 999999,
      "result": "NOT_FOUND",
      "status": null,
      "message": "La ruta no existe."
    }
  ]
}
  
```

1 616 → SUCCESS / EXECUTED

2 192 → ERROR: ya no estaba READY

3 999999 → NOT_FOUND

La solicitud procesó tres identificadores de forma independiente: confirmó una ejecución, rechazó una ruta por estado y reportó un ID inexistente. Los intentos SUCCESS y ERROR quedaron registrados en logs.

Validaciones automatizadas aprobadas

Resultados obtenidos en Codespaces

17 backend

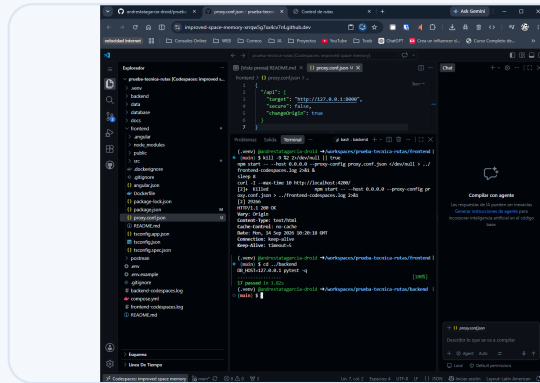
2 frontend

build OK

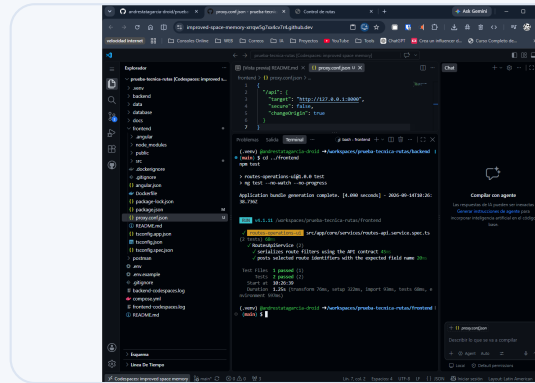
13 Postman

13 SQL PASS

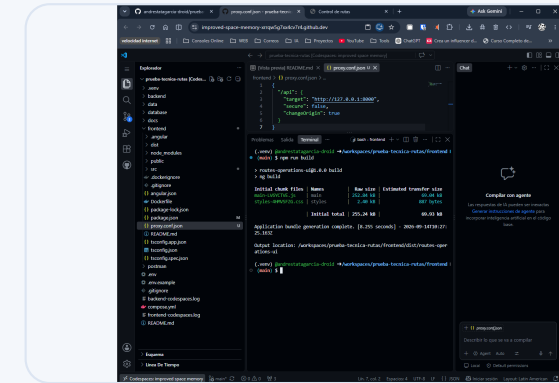
5.000 controladas



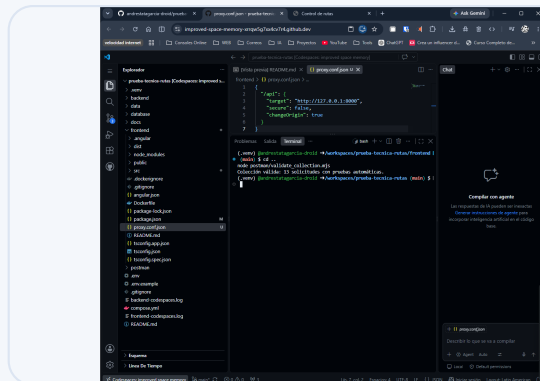
Backend · 17 passed



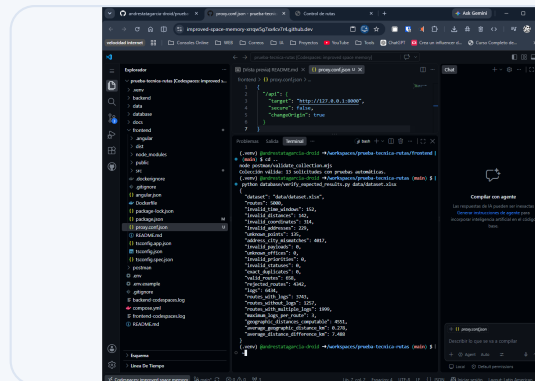
Frontend · 2 passed



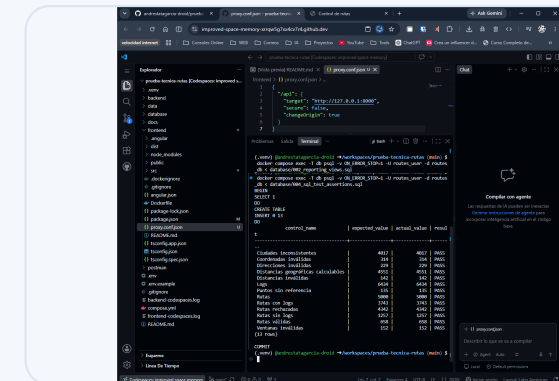
Build de producción



Colección Postman



Controles independientes



Aserciones SQL

Resultado integral de la verificación

Alcance comprobado del MVP

1 Separación de datos confirmada

Staging conservó la evidencia original y las tablas operacionales recibieron únicamente rutas válidas.

2 Calidad reproducida

Los controles independientes y SQL coincidieron en 658 rutas válidas y 4.342 rechazadas.

3 Ejecución protegida

El estado READY, la validación del backend y el bloqueo transaccional controlaron cada transición.

4 Alcance del MVP identificado

El sistema cubrió el flujo solicitado. Servidor productivo, tareas asíncronas, observabilidad y CI/CD quedaron como evolución.

“La solución no solo procesa rutas: conserva el origen, hace explícita la calidad, protege la operación y deja evidencia verificable de cada decisión.”